

Informe de Laboratorio 04

Tema: Programación Distribuida en Java con RMI (Invocación Remota de Métodos)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	<i>Programación Distribuida en Java con RMI (Invocación Remota de Métodos)</i>				
NÚMERO DE PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
FECHA DE PRESENTACIÓN	08/05/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): • Chambilla Perca Ricardo • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

1. EJERCICIOS/PROBLEMAS PROPUESTOS

- **Ejercicio 1:** Analizar, implementar, ejecutar y probar el código del ejercicio de Ejemplo de RMI en Java (Medicinas / Farmacia).
- **Ejercicio 2:** Utilizando RMI desarrollar una aplicación que permita realizar un Sistema simple de tarjetas de crédito.
- **Ejercicio 3:** Utilizando RMI para un Servicio remoto de conversión de moneda (Soles a Dólares y Euros).

2. Desarrollo de los Ejercicios

2.1. Ejercicio 1: Farmacia RMI (Medicinas)

Se implementó el código tal como se propuso en el Anexo 1 de la guía.

- **StockInterface.java y MedicineInterface.java:** Definen los métodos que estarán expuestos para ser llamados remotamente, heredando de `java.rmi.Remote`.
- **Stock.java:** Es la implementación del servidor que gestiona el inventario, permitiendo comprar medicinas (`buyMedicine`) y listar el stock (`getStockProducts`). Extiende `UnicastRemoteObject`, lo que significa que de este objeto se generarán *stubs* para la comunicación remota.

- **ServerSide.java:** Instancia nuestro objeto **Stock** y lo publica en el RMI Registry usando **Naming.rebind("PHARMACY", pharmacy)**.
- **ClienteSide.java:** Busca el objeto remoto publicado mediante **Naming.lookup("PHARMACY")** y llama a sus métodos como si fueran locales.
- **StockException.java:** Manejo de excepciones propio del dominio (por ejemplo cuando no hay stock).

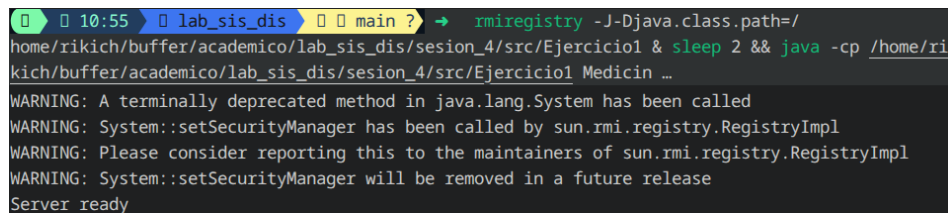
Compilación y Ejecución Automatizada:

Listing 1: Pasos de ejecución

```

1 # 1. Compilar los archivos Java
2 cd src/Ejercicio1
3 javac Medicinas/*.java
4
5 # 2. Levantar RMI Registry y el Servidor (en background)
6 rmiregistry -J-Djava.class.path=./ &
7 java -cp ./ Medicinas.ServerSide
8 # (Debe mostrar: Server ready)
9
10 # 3. Correr el Cliente
11 java -cp ./ Medicinas.ClienteSide

```



```

10:55 lab_sis_dis main ? → rmiregistry -J-Djava.class.path=/
home/rikich/buffer/academico/lab_sis_dis/sesion_4/src/Ejercicio1 & sleep 2 && java -cp /home/ri
kich/buffer/academico/lab_sis_dis/sesion_4/src/Ejercicio1 Medicin ...
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
Server ready

```

Figura 1: Ejecución exitosa del sistema RMI de Medicinas (Ejercicio 1).

2.2. Ejercicio 2: Tarjetas de Crédito (Credito)

Sistema simple para gestionar el saldo de tarjetas de crédito a través de diferentes clientes.

- **CreditCardInterface.java:** Interfaz remota que define operaciones bancarias básicas: consultar el saldo (**checkBalance**), procesar un pago y reducir crédito (**processPayment**), y una función auxiliar para que el servidor cree tarjetas (**registerCard**).
- **CreditCardServerImpl.java:** Implementación remota. Internamente utiliza un **HashMap<String, Double>** para hacer el seguimiento del límite disponible para cada número de tarjeta en memoria. Revisa que el monto a cobrar no exceda el límite e implementa la resta.
- **ServerSide.java:** Registra instancias de tarjetas ficticias en el servidor y luego publica el servidor mismo en el registro con el nombre **CREDIT_CARD_SERVICE**.
- **ClienteSide.java:** Implementa un menú iterativo para el usuario, que busca el objeto en RMI y llama pasándole un número de tarjeta y monto a las funciones remotas para verificar o restar saldos.

```

14:38 lab_sis_dis main !?+2 java -cp /home/rikich/buffer/academico/lab_sis
_dis/sesion_4/src/Ejercicio2 Credito.ClientSide

Ingresa la opcion
1: Consultar Saldo
2: Realizar Pago
3: Salir

2
Ingresa numero de tarjeta: 1234567890
Ingresa monto a pagar: 800
Pago realizado con exito.

Ingresa la opcion
1: Consultar Saldo
2: Realizar Pago
3: Salir

1
Ingresa numero de tarjeta: 1234567890
Saldo disponible: 201.0

```

Figura 2: Operaciones de consulta y pago en el sistema de Tarjetas de Crédito.

2.3. Ejercicio 3: Conversor de Moneda (Conversion)

Servicio remoto simple para conversión de soles.

- **ConvertorInterface.java:** Tal como se solicitó, incluye los 2 métodos remotos necesarios: `convertirADolares(double monto)` y `convertirAEuros(double monto)`.
- **ConvertorImpl.java:** Implementa la interfaz y define la lógica de negocio; es decir, multiplica el monto provisto por una tasa fija estática configurada como un `final double` para retornar el valor de cambio.
- **ServerSide.java:** Expone el servicio de conversión en el registro con el nombre `CONVERSION_SERVICE`.
- **ClientSide.java:** Un programa por consola que consume el servicio, recibe ingresos en Soles por teclado e invoca directamente los cálculos en el servidor para obtener los valores convertidos.

```

14:13 lab_sis_dis main !?+2 java -cp /home/rikich/buffer/academico/lab_sis
_dis/sesion_4/src/Ejercicio3 Conversion.ClientSide

Ingresa la opcion para convertir SOLES a:
1: Dolares
2: Euros
3: Salir

2
Ingresa monto en soles: 3
Equivalente en euros: 0.75

Ingresa la opcion para convertir SOLES a:
1: Dolares
2: Euros
3: Salir

```

Figura 3: Prueba del cliente conversor de monedas consultando al servidor RMI.

3. CUESTIONARIO

1. ¿Cómo funciona el registro RMI?

El registro RMI (RMI Registry) actúa como un servicio de nombres o directorio central. Los servidores registran sus objetos remotos en este servicio asociándolos con un nombre lógico único (usando `Naming.rebind` o `Naming.bind`). Posteriormente, los clientes se conectan a este registro, buscan el nombre lógico (usando `Naming.lookup`) y obtienen un *stub* (una referencia o proxy) que les permite invocar métodos del objeto remoto como si estuviera en su propia máquina virtual.

2. ¿Cuáles son las subclases para soportar carga dinámica de clases?

RMI soporta la carga dinámica a través de la clase `RMIClassLoader`. Esta clase permite descargar dinámicamente el bytecode de las clases (como stubs, skeletons o parámetros) desde una URL remota (un codebase) si no se encuentran en el classpath local del cliente o servidor.

3. ¿Qué ventajas y desventajas presenta RMI frente a la comunicación con sockets?

- **Ventajas:** Proporciona un nivel de abstracción mucho mayor, permitiendo llamar a métodos remotos con la misma sintaxis que llamadas locales. Maneja automáticamente la serialización de objetos (paso de parámetros complejos y retornos) y la sincronización, evitando tener que parsear bytes puros a mano como en sockets.
- **Desventajas:** Está fuertemente acoplado al lenguaje Java. Además, presenta una sobrecarga (overhead) en el rendimiento debido al proceso de serialización/deserialización y las capas intermedias, siendo más lento que la comunicación directa por sockets crudos.

4. ¿Por qué RMI necesita que los objetos sean serializables y cómo se podría escalar este sistema a múltiples servidores RMI?

RMI necesita que los objetos implementen la interfaz `Serializable` porque para transmitir los parámetros o valores de retorno a través de la red, los objetos en memoria deben ser convertidos en un flujo de bytes (byte stream) y reconstruidos al otro lado. Para escalar el sistema a múltiples servidores RMI, se puede implementar un balanceador de carga que redirija las peticiones iniciales del cliente, utilizar un cluster de RMI Registries, o migrar a arquitecturas empresariales (como JNDI, EJBs) o microservicios modernos donde varios nodos servidor pueden registrar réplicas del servicio.

5. ¿Qué medidas de seguridad se deberían considerar para invocaciones remotas en entornos reales?

En entornos reales se debe:

- Cifrar la comunicación envolviendo RMI sobre SSL/TLS para evitar el *sniffing* de datos.
- Implementar mecanismos de autenticación y autorización antes de permitir la ejecución de métodos críticos.
- Configurar políticas de seguridad estrictas (tradicionalmente con `SecurityManager` y `policy files`, limitando permisos de red y acceso a archivos).
- Validar rigurosamente los datos de entrada para evitar ataques por deserialización maliciosa.

4. Referencias

- Tanenbaum, A.S. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
- Ceballos, F. J. (2006). *Java 2, Curso de programación*. México: Alfaomega, RaMa.
- Deitel, H. M., & Deitel, P. J. (2004). *Cómo programar en Java*. México: Pearson Educación.
- García Tomás, J., Ferrando, S., & Piattini, M. (2001). *Redes para procesos distribuidos*. México: Alfaomega Ra-Ma.
- Tutorial oficial de RMI: <https://docs.oracle.com/javase/tutorial/rmi/>
- Oracle Java RMI Documentation: <https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/>